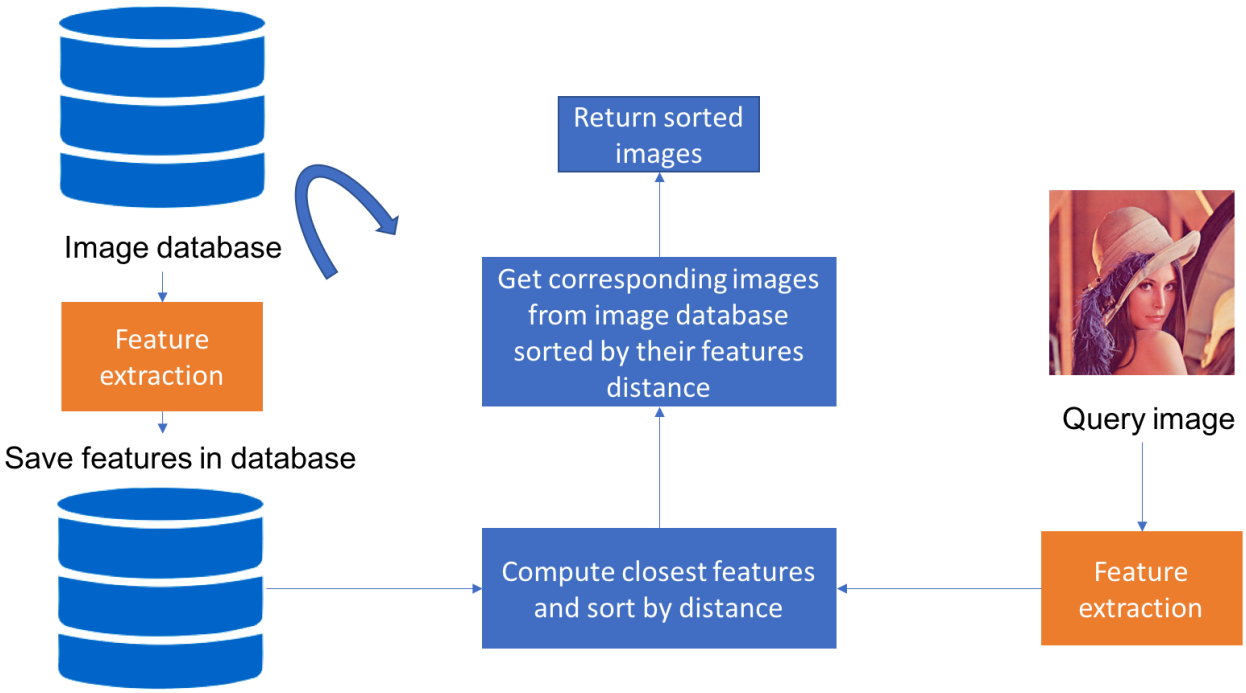
Objectif

La recherche d'images basée sur le contenu (CBIR) est une méthode permettant de rechercher des images à partir des caractéristiques visuelles, c'est-à-dire induites par les pixels. L'utilisation la plus connue de cette méthode est la fonction de recherche d'images du moteur de recherche Google. Elle s'effectue en quatre étapes :

1. Extraction des caractéristiques d'une base de données d'images pour former une base de données des caractéristiques,
2. Extraction des caractéristiques de l'image fournie en entrée,
3. Recherche des caractéristiques les plus similaires dans la base de données,
4. Retour de l'image associée aux caractéristiques trouvées.



Cette méthode répose sur l'algorithme d'extraction et la comparaison des caractéristiques.

Mesure de similarité

La recherche est exacte (force brute) : les écarts viennent des vecteurs et de la mesure, pas d'un index approximatif.

- Distance cosinus : 1 - similarité cosinus, ne compare que les directions
- Distance de manhattan

- Distance euclidienne

Algorithme d'extraction

- AKAZE et ORB, sous forme de sac de mots visuels : vocabulaire de 512 mots appris sur la base d'images, affectation des descripteurs binaires par distance de Hamming
- VGG16, couche `fc2` (4096)
- NasNet Large, moyenne globale (4032)
- EfficientNet B7, moyenne globale (2560)

L'objectif de trouver la bonne combinaisons (algorithme d'extraction & mesure de similarité) qui permet d'avoir des réponses pertinentes.

Jeux de données

Nous avons utilisé la version 6 du jeu de données Apparel pour notre évaluation, découpé en une base d'images (`train` , 67 %) et des requêtes (`test` , 33 %), stratifiés par catégorie.

```
Elle contient 11385 images réparties en 24 catégories, divisées en deux jeux :
- base d'images : 7627 images
- requêtes : 3758 images
```

Evaluation

Couverture des extracteurs

Toutes les méthodes reçoivent les mêmes images. Une image sans vecteur (aucun point clé détecté) reste dans l'évaluation : absente de la base, elle ne peut pas être retournée ; requête, elle obtient un score nul.

method	base_sans_vecteur	requetes_sans_vecteur
AKAZE	4	4
ORB	0	0
VGG	0	0
NasNet	0	0
EfficientNet	0	0

Nombre d'images sans vecteur par méthode, dans la base et parmi les requêtes.

Métriques d'évaluation

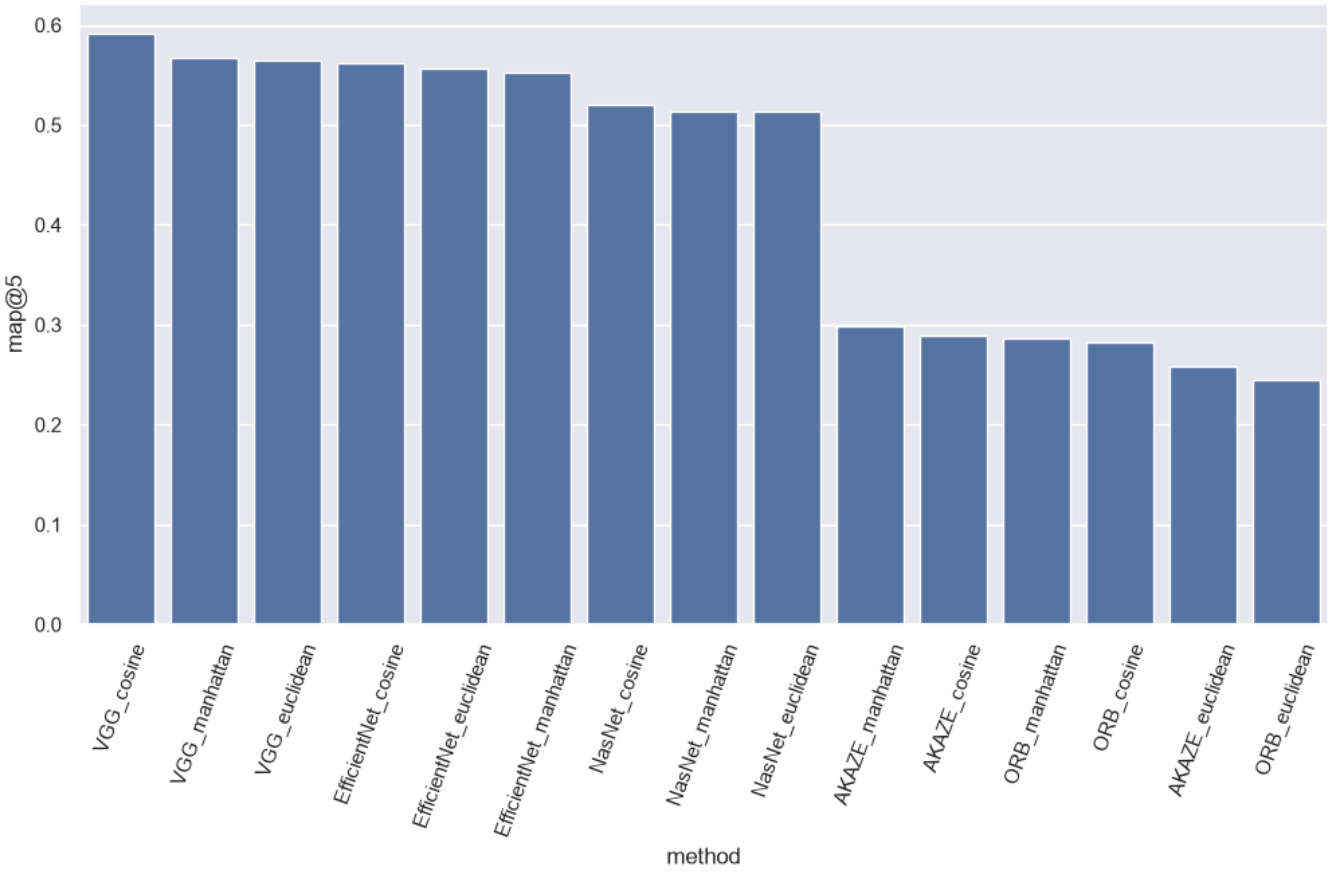
Chaque requête retourne ses 5 plus proches images.

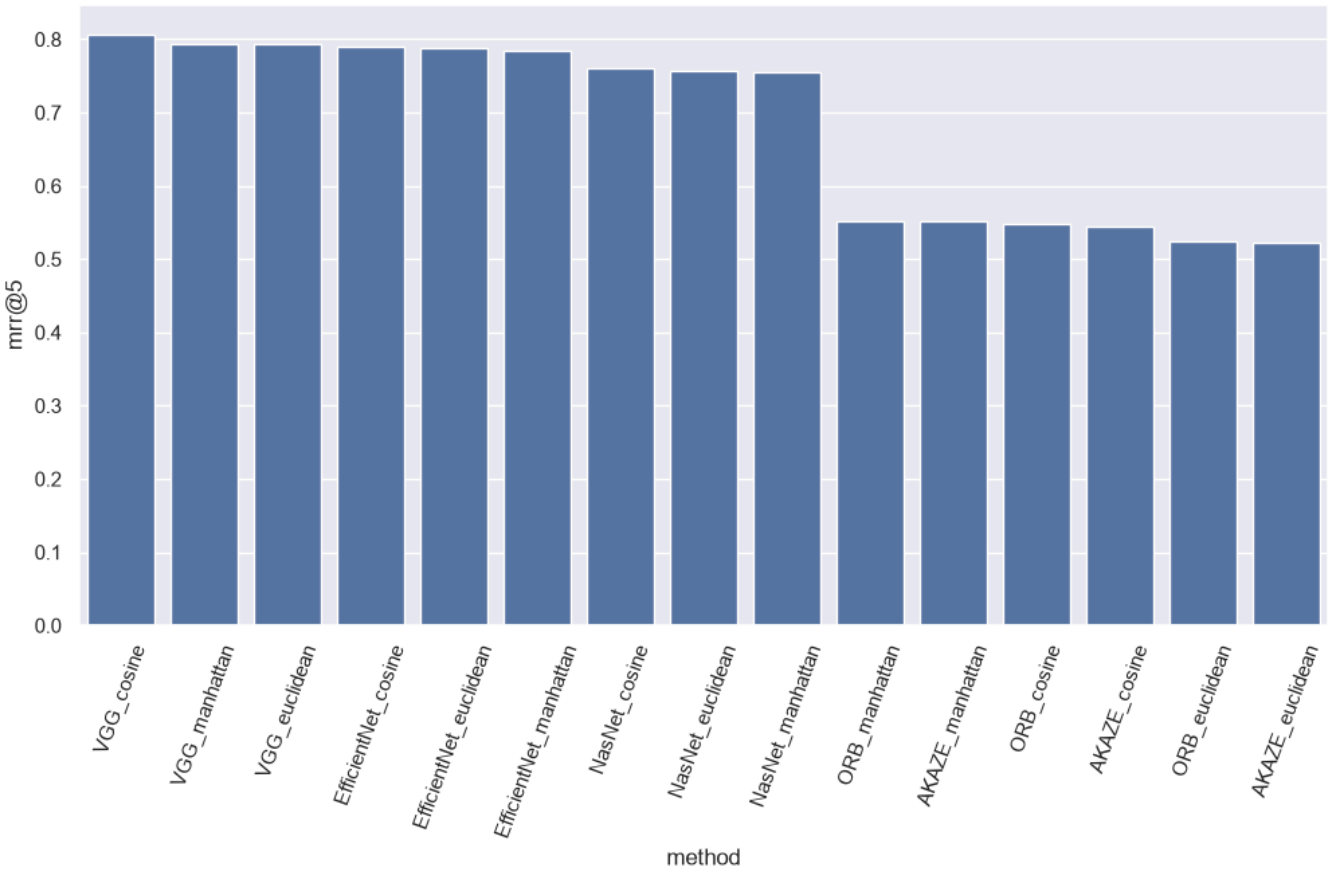
- MAP@5 : précision moyenne sur les 5 résultats, divisée par `min(images pertinentes dans la base, 5)`
- MRR@5 : inverse du rang du premier résultat pertinent, 0 au-delà du rang 5
- Précision au premier rang
- Temps d'une requête (ms) : temps moyen d'extraction d'une image seule + temps moyen de recherche

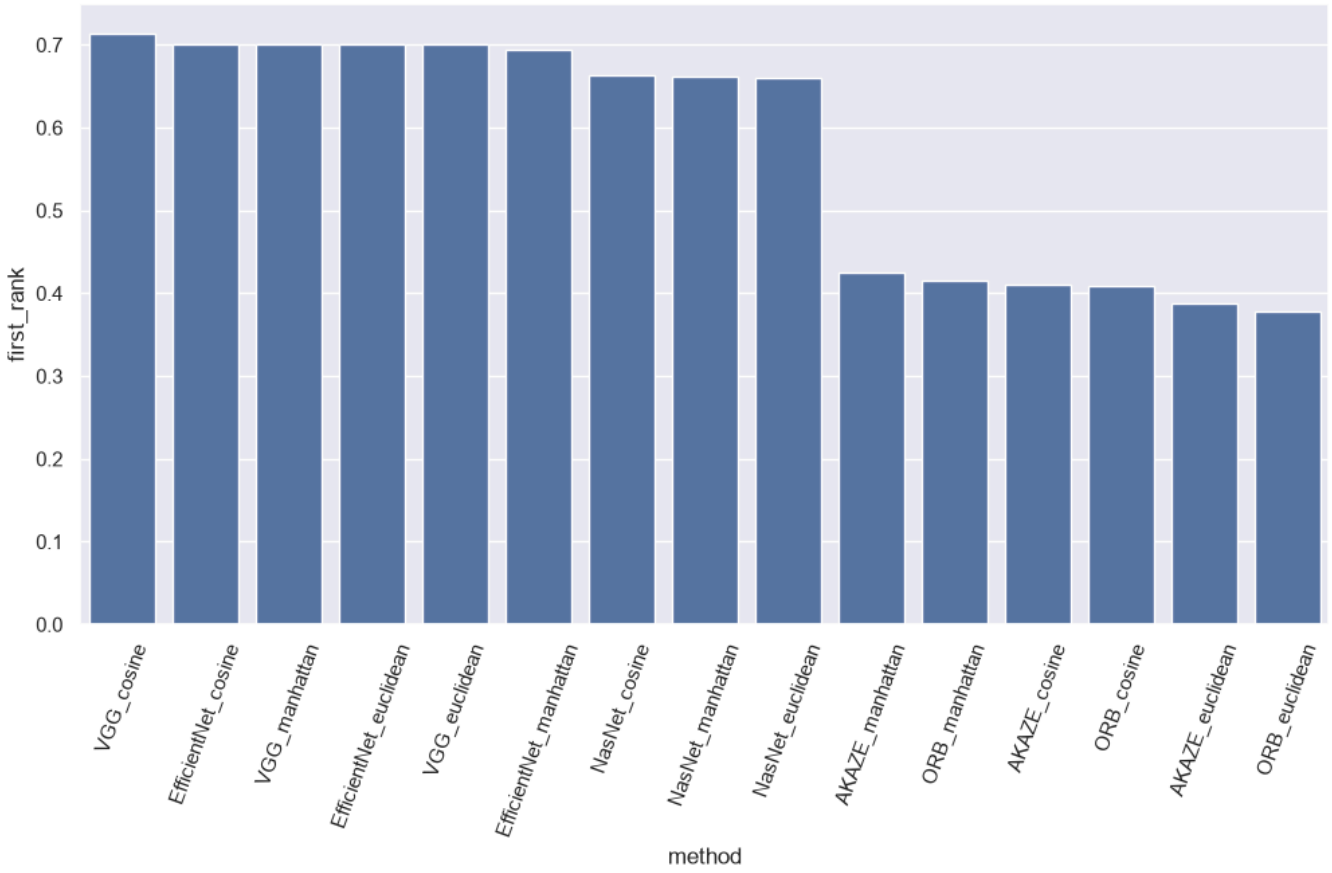
Pour plus de détail voir

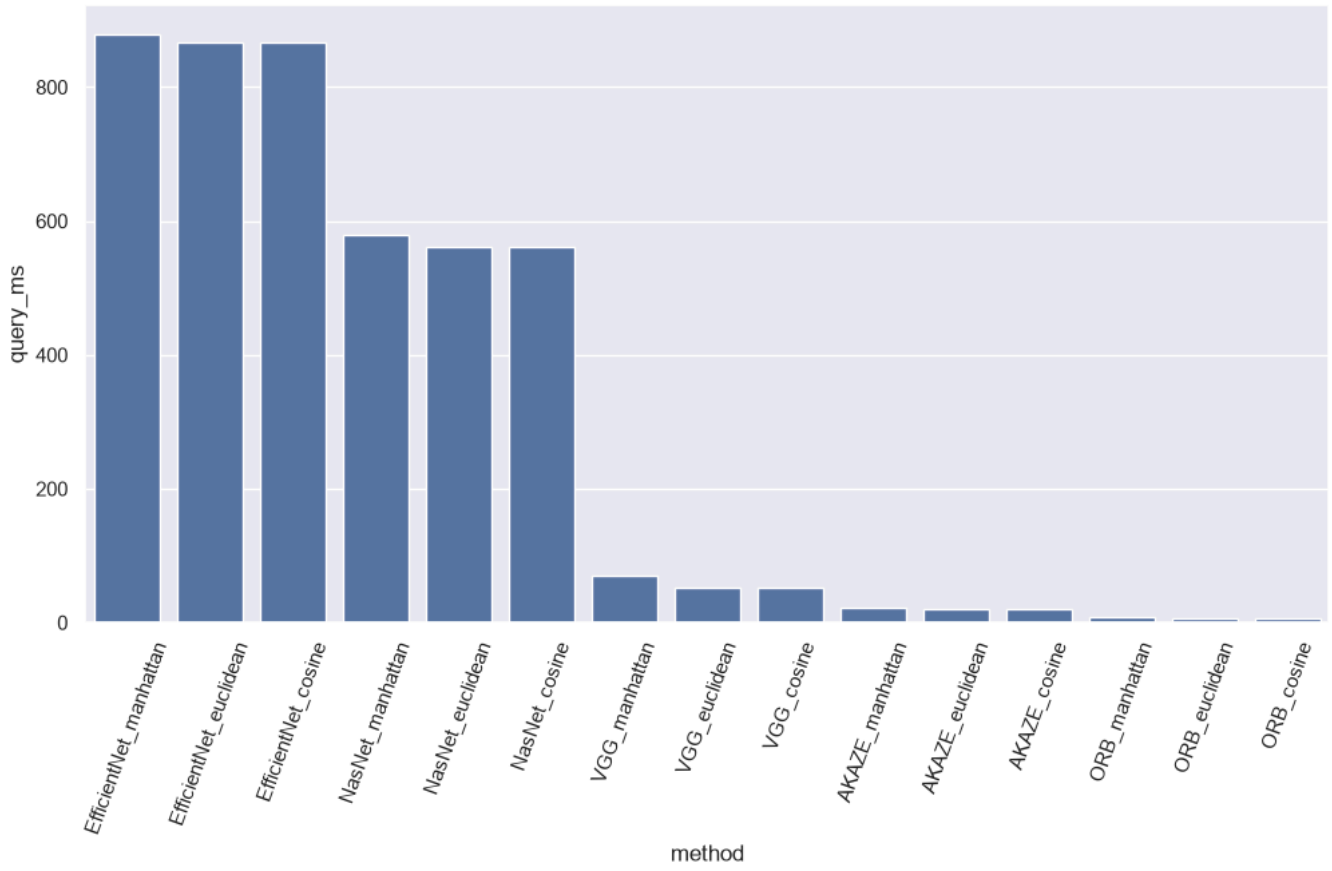
Colors

method	map@5	mrr@5	first_rank	query_ms
AKAZE_cosine	0.28856	0.543831	0.410857	19.695101
AKAZE_euclidean	0.258061	0.522317	0.38744	19.701567
AKAZE_manhattan	0.298627	0.551597	0.425226	21.518462
ORB_cosine	0.281823	0.548599	0.40926	5.591079
ORB_euclidean	0.245385	0.523479	0.377861	5.608884
ORB_manhattan	0.287037	0.552621	0.415647	7.353868
VGG_cosine	0.591204	0.806151	0.713411	52.552482
VGG_euclidean	0.564672	0.793986	0.700639	52.806309
VGG_manhattan	0.5678	0.794332	0.701171	69.171145
NasNet_cosine	0.520162	0.760316	0.663651	561.878157
NasNet_euclidean	0.512804	0.756302	0.660724	562.096291
NasNet_manhattan	0.513866	0.755251	0.661522	579.608441
EfficientNet_cosine	0.562177	0.789179	0.701437	867.427039
EfficientNet_euclidean	0.556772	0.787768	0.700905	867.572621
EfficientNet_manhattan	0.552614	0.783524	0.693986	878.440194


map@5
0.0
0.1
0.2
0.3
0.4
0.5
0.6
VGG_cosine
VGG_manhattan
VGG_euclidean
EfficientNet_cosine
EfficientNet_euclidean
EfficientNet_manhattan
NasNet_cosine
NasNet_manhattan
NasNet_euclidean
AKAZE_manhattan
AKAZE_cosine
ORB_manhattan
ORB_cosine
AKAZE_euclidean
ORB_euclidean
method

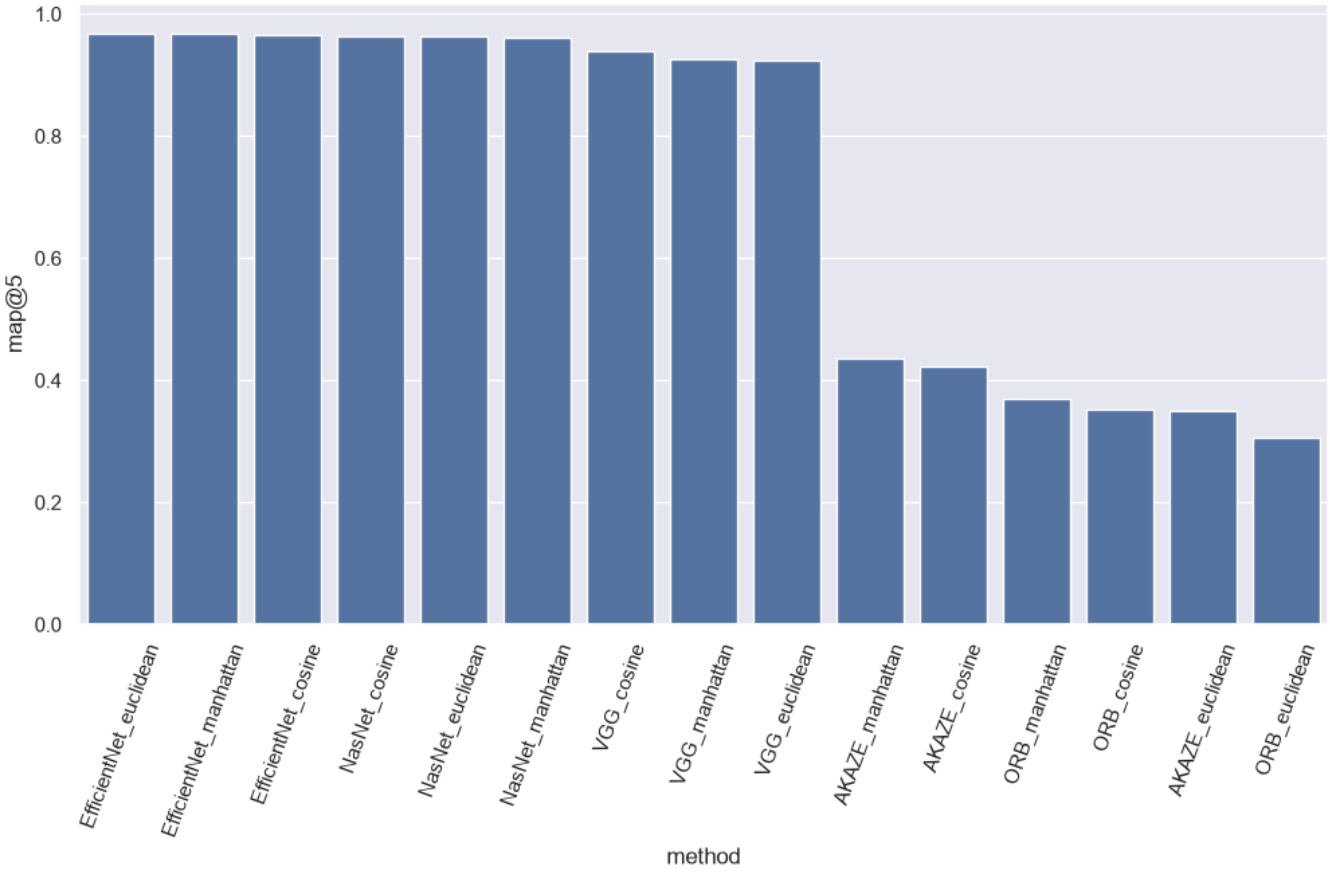

mrr@5
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
VGG_cosine
VGG_manhattan
VGG_euclidean
EfficientNet_cosine
EfficientNet_euclidean
EfficientNet_manhattan
NasNet_cosine
NasNet_euclidean
NasNet_manhattan
ORB_manhattan
AKAZE_manhattan
ORB_cosine
AKAZE_cosine
ORB_euclidean
AKAZE_euclidean
method

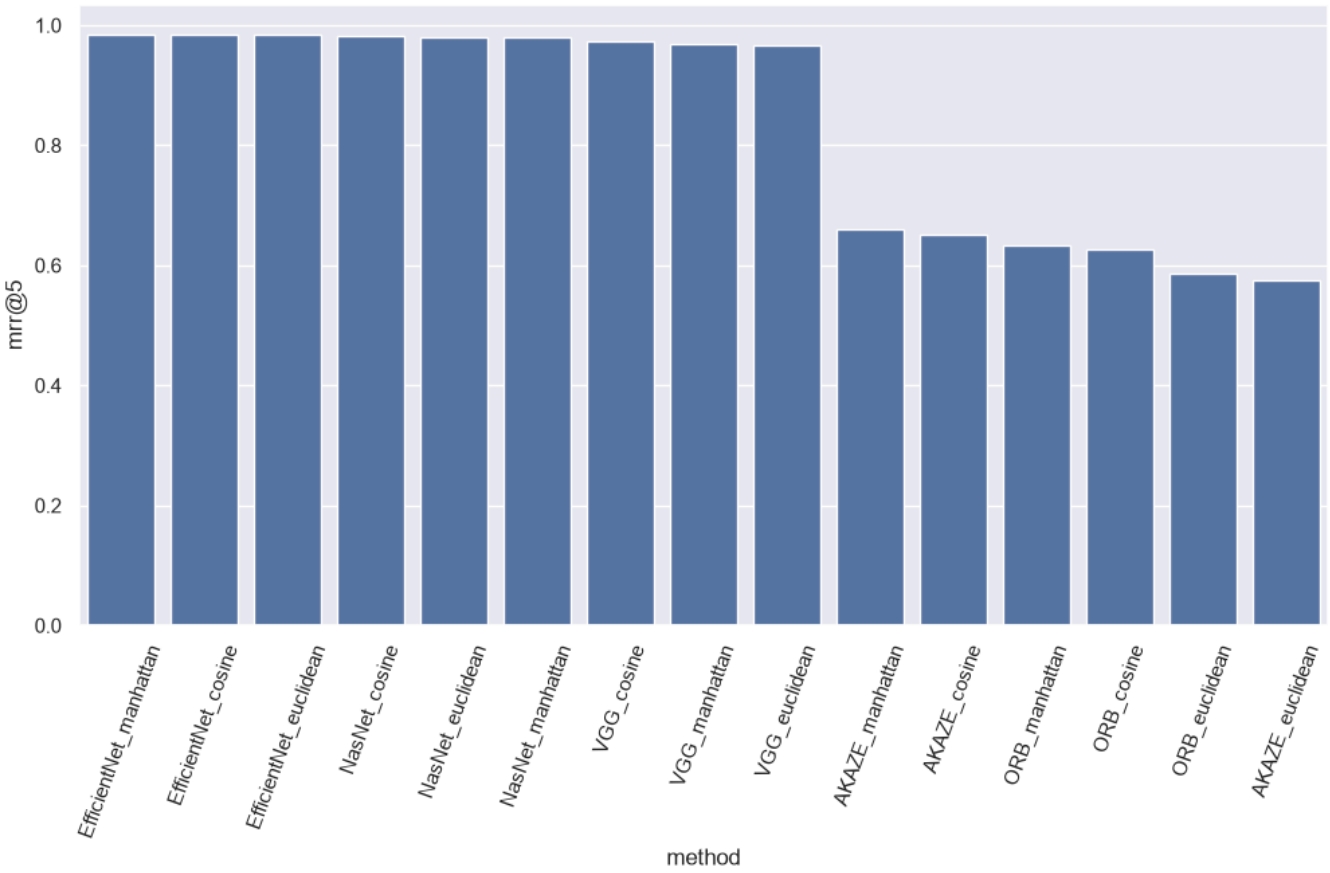

first_rank
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
VGG_cosine
EfficientNet_cosine
VGG_manhattan
EfficientNet_euclidean
VGG_euclidean
EfficientNet_manhattan
NasNet_cosine
NasNet_manhattan
NasNet_euclidean
AKAZE_manhattan
ORB_manhattan
AKAZE_cosine
ORB_cosine
AKAZE_euclidean
ORB_euclidean
method

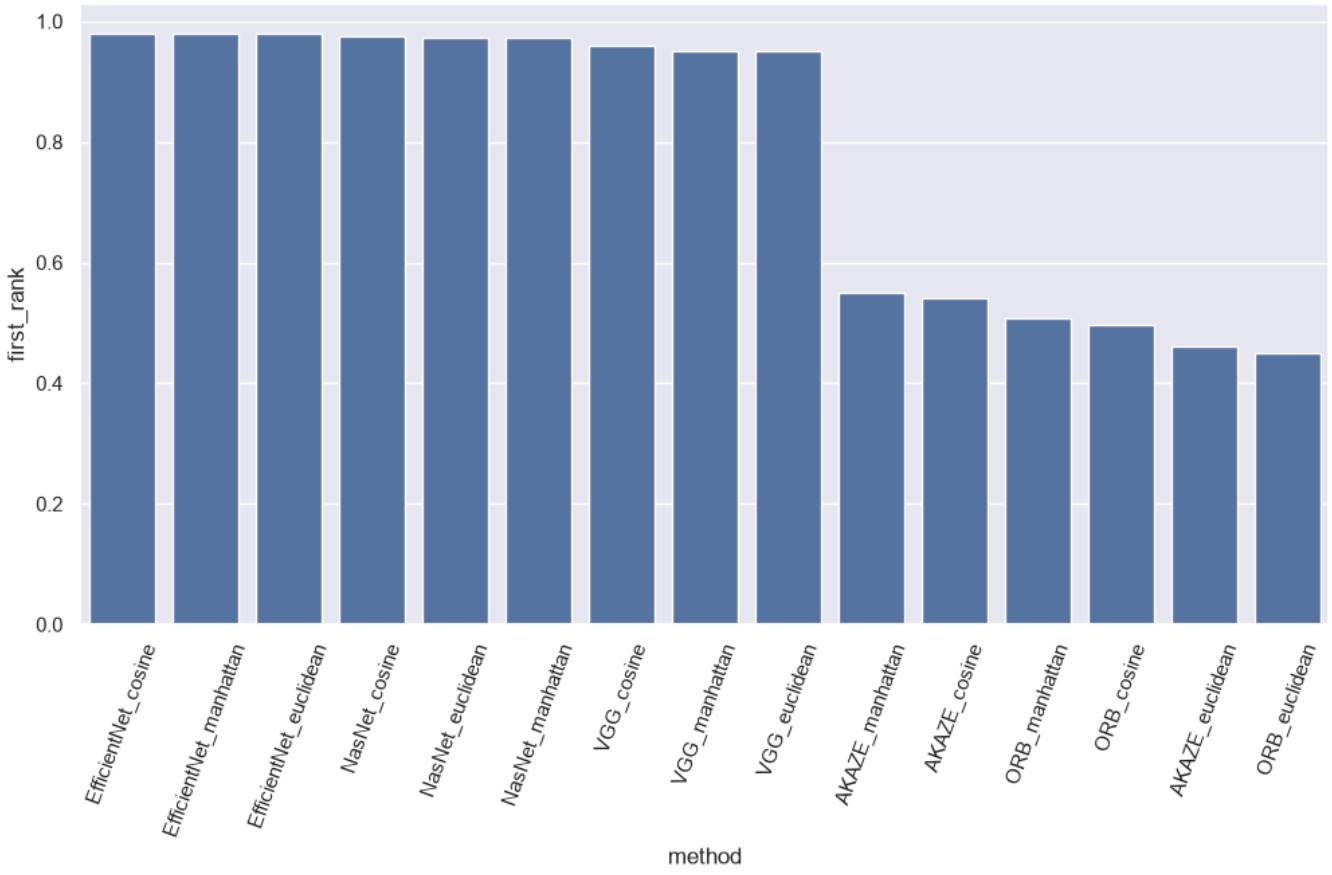

query_ms
0
200
400
600
800
EfficientNet_manhattan
EfficientNet_euclidean
EfficientNet_cosine
NasNet_manhattan
NasNet_euclidean
NasNet_cosine
VGG_manhattan
VGG_euclidean
VGG_cosine
AKAZE_manhattan
AKAZE_euclidean
AKAZE_cosine
ORB_manhattan
ORB_euclidean
ORB_cosine
method

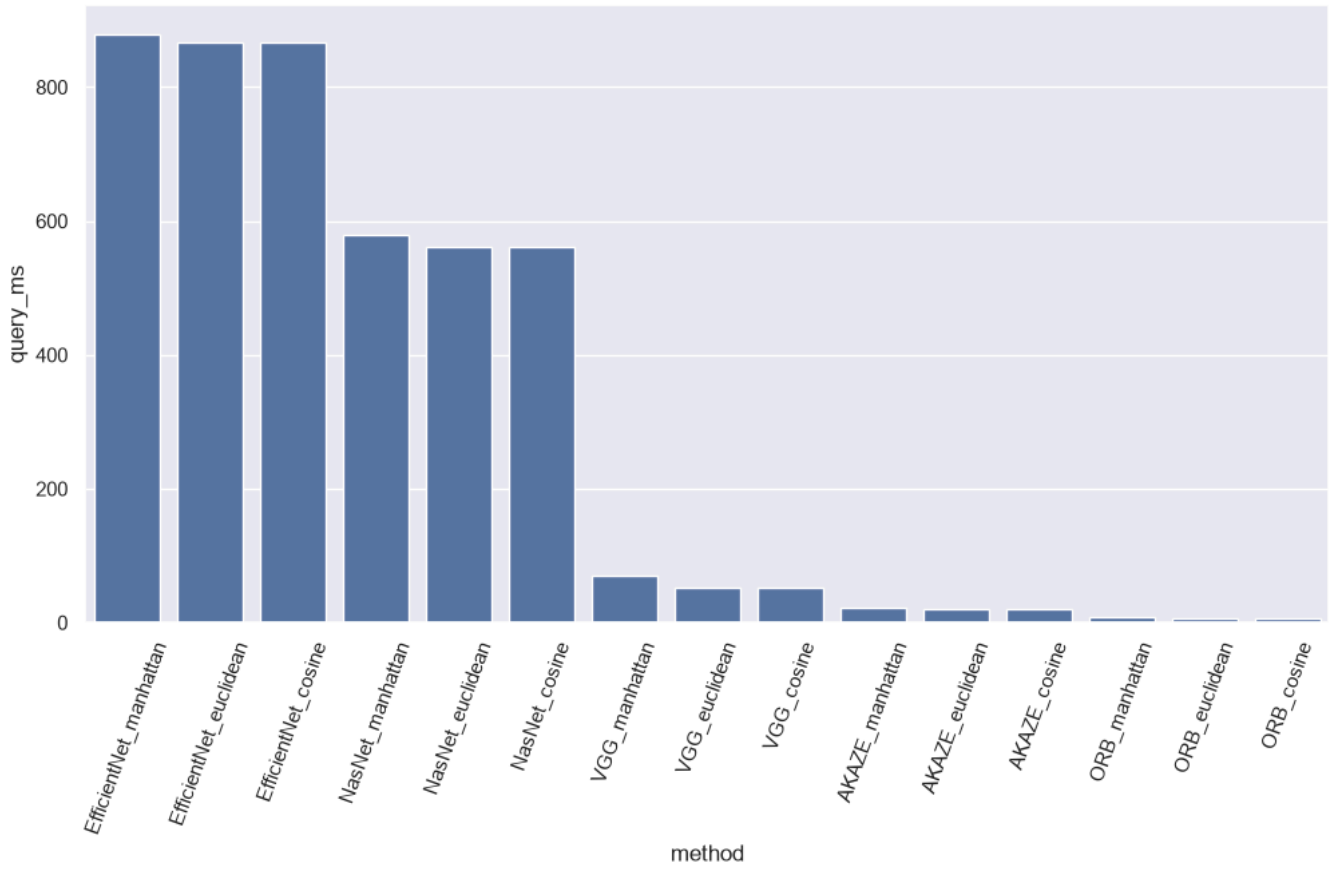
Styles

method	map@5	mrr@5	first_rank	query_ms
AKAZE_cosine	0.42334	0.650914	0.542044	19.695101
AKAZE_euclidean	0.350433	0.575993	0.460085	19.701567
AKAZE_manhattan	0.435774	0.659575	0.549494	21.518462
ORB_cosine	0.351819	0.626348	0.497605	5.591079
ORB_euclidean	0.306	0.586872	0.450506	5.608884
ORB_manhattan	0.370456	0.633515	0.507185	7.353868
VGG_cosine	0.940061	0.972317	0.959553	52.552482
VGG_euclidean	0.923888	0.966951	0.949707	52.806309
VGG_manhattan	0.926197	0.967758	0.95157	69.171145
NasNet_cosine	0.963434	0.982078	0.975253	561.878157
NasNet_euclidean	0.962743	0.980247	0.973656	562.096291
NasNet_manhattan	0.961667	0.980091	0.973656	579.608441
EfficientNet_cosine	0.966463	0.98403	0.979244	867.427039
EfficientNet_euclidean	0.967596	0.983679	0.978712	867.572621
EfficientNet_manhattan	0.967563	0.984078	0.979244	878.440194


map@5
1.0
0.8
0.6
0.4
0.2
0.0
EfficientNet_euclidean
EfficientNet_manhattan
EfficientNet_cosine
NasNet_cosine
NasNet_euclidean
NasNet_manhattan
VGG_cosine
VGG_manhattan
VGG_euclidean
AKAZE_manhattan
AKAZE_cosine
ORB_manhattan
ORB_cosine
AKAZE_euclidean
ORB_euclidean
method

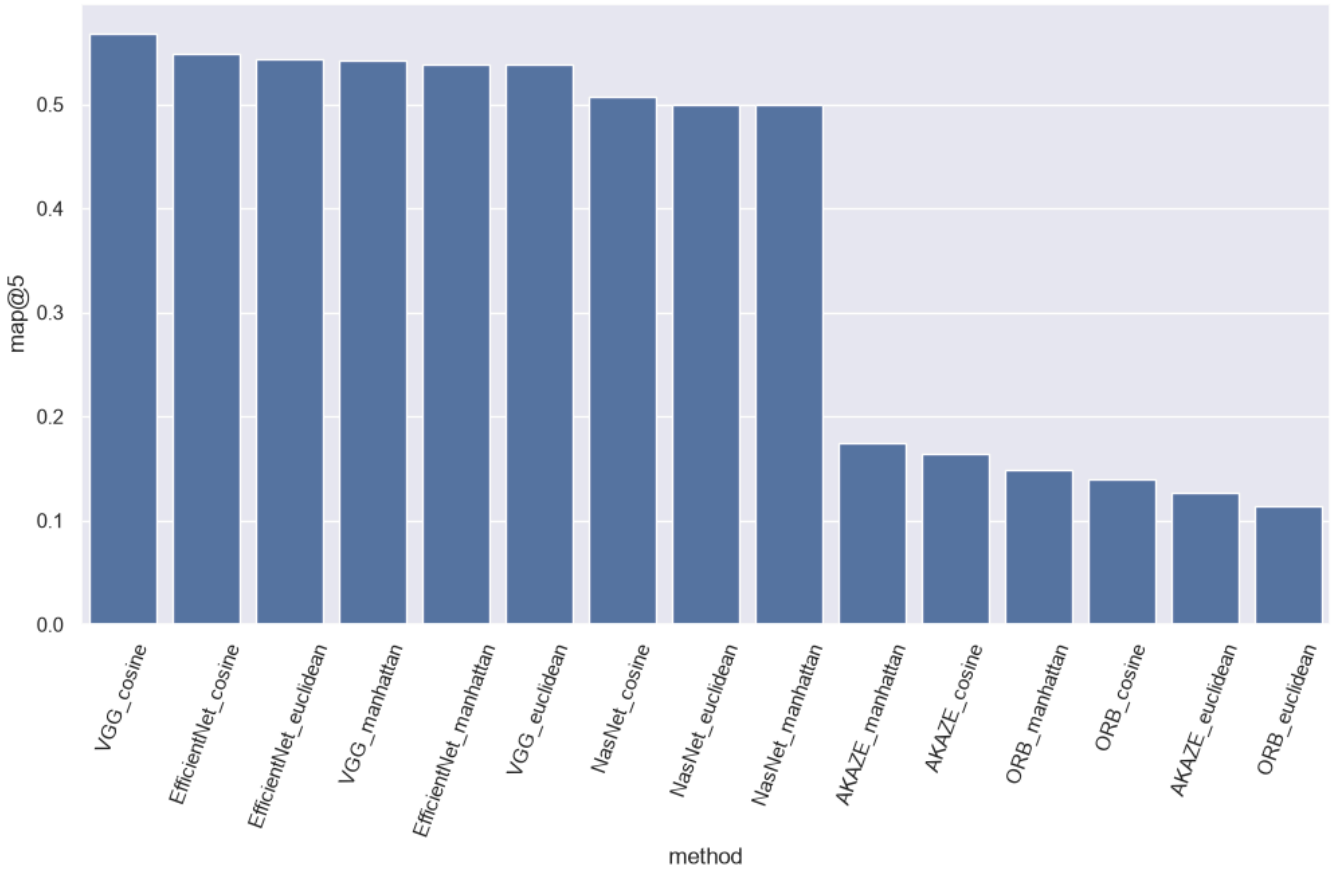

mrr@5
1.0
0.8
0.6
0.4
0.2
0.0
EfficientNet_manhattan
EfficientNet_cosine
EfficientNet_euclidean
NasNet_cosine
NasNet_euclidean
NasNet_manhattan
VGG_cosine
VGG_manhattan
VGG_euclidean
AKAZE_manhattan
AKAZE_cosine
ORB_manhattan
ORB_cosine
ORB_euclidean
AKAZE_euclidean
method

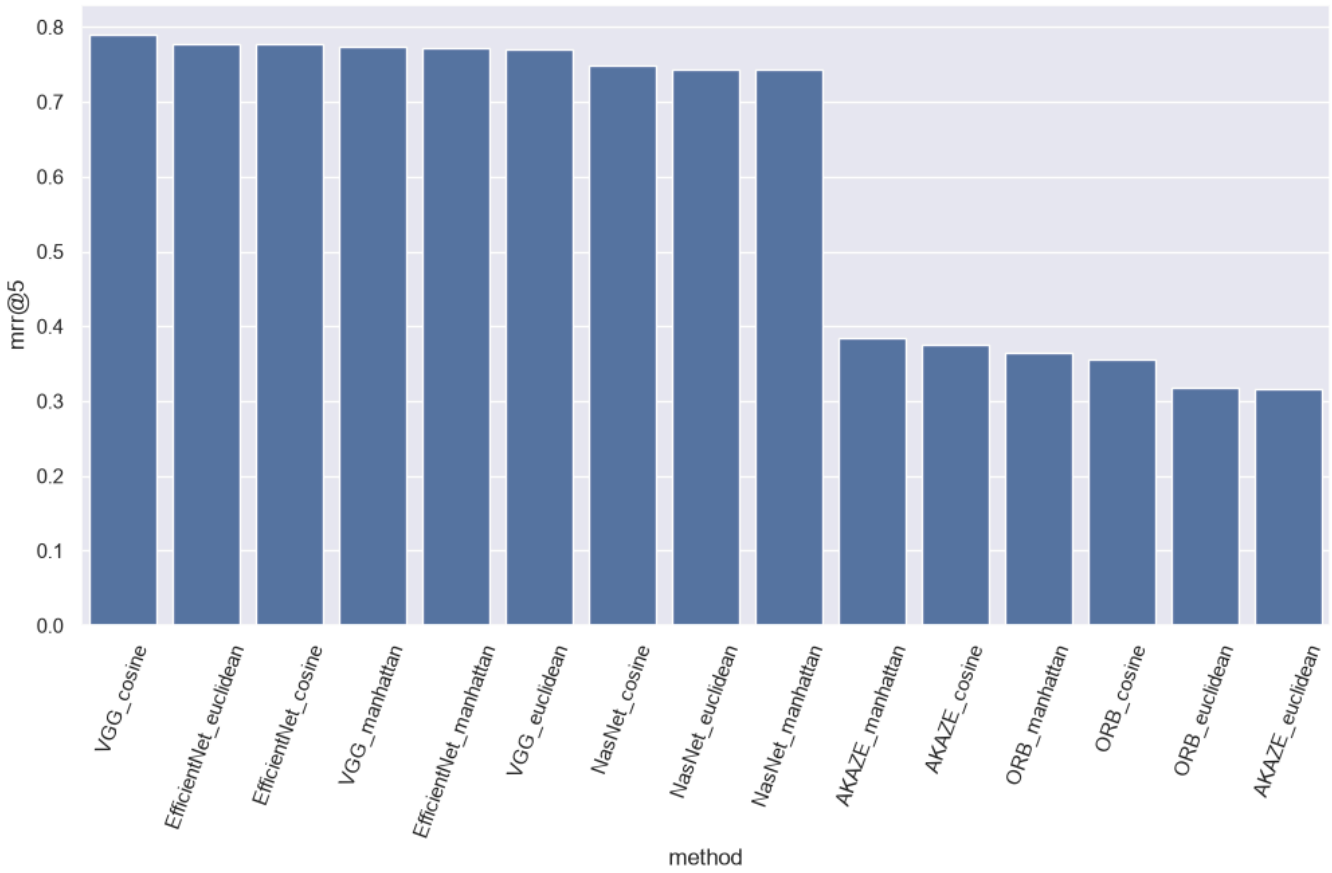

first_rank
1.0
0.8
0.6
0.4
0.2
0.0
EfficientNet_cosine
EfficientNet_manhattan
EfficientNet_euclidean
NasNet_cosine
NasNet_euclidean
NasNet_manhattan
VGG_cosine
VGG_manhattan
VGG_euclidean
AKAZE_manhattan
AKAZE_cosine
ORB_manhattan
ORB_cosine
AKAZE_euclidean
ORB_euclidean
method

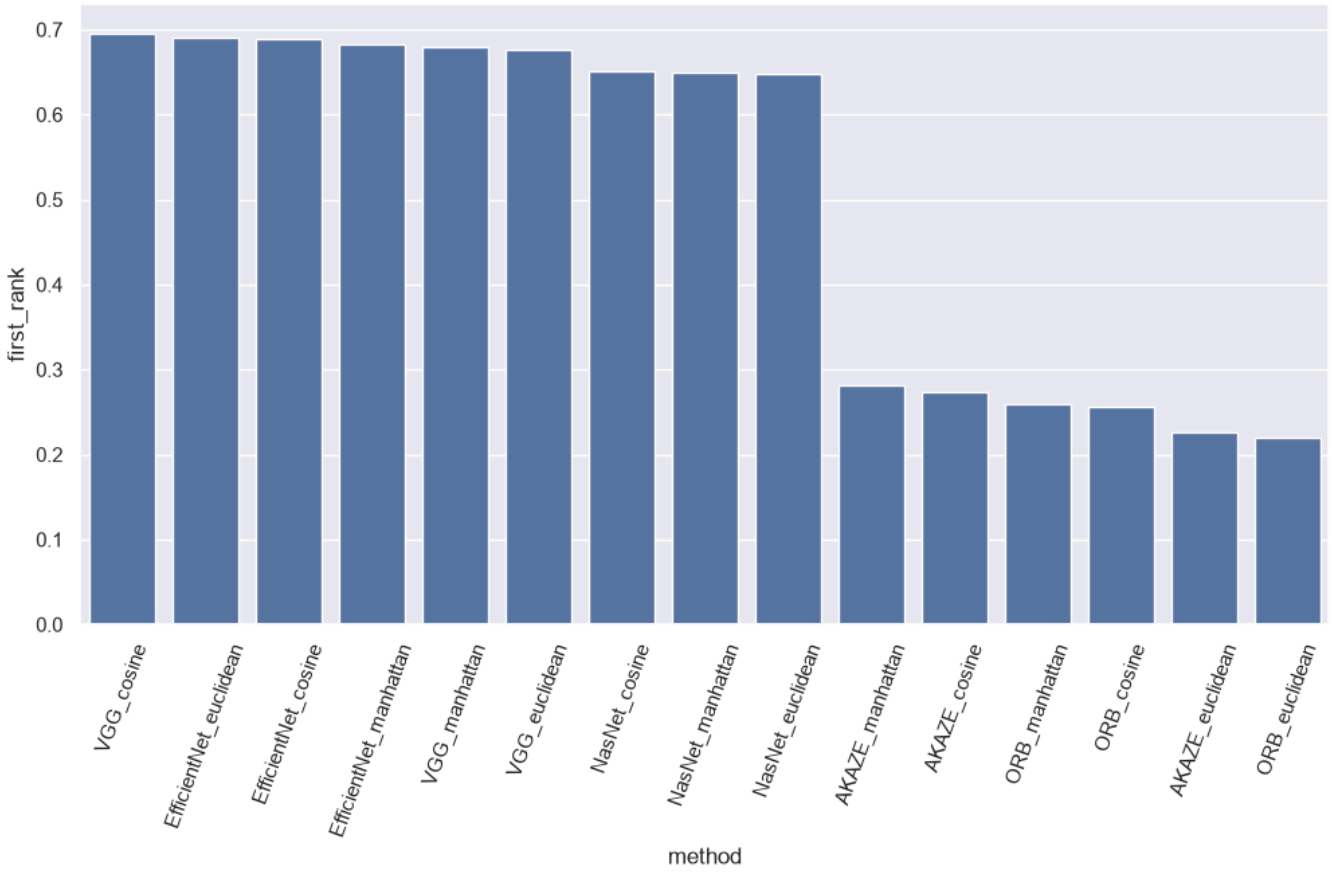

query_ms
800
600
400
200
0
EfficientNet_manhattan
EfficientNet_euclidean
EfficientNet_cosine
NasNet_manhattan
NasNet_euclidean
NasNet_cosine
VGG_manhattan
VGG_euclidean
VGG_cosine
AKAZE_manhattan
AKAZE_euclidean
AKAZE_cosine
ORB_manhattan
ORB_euclidean
ORB_cosine
method

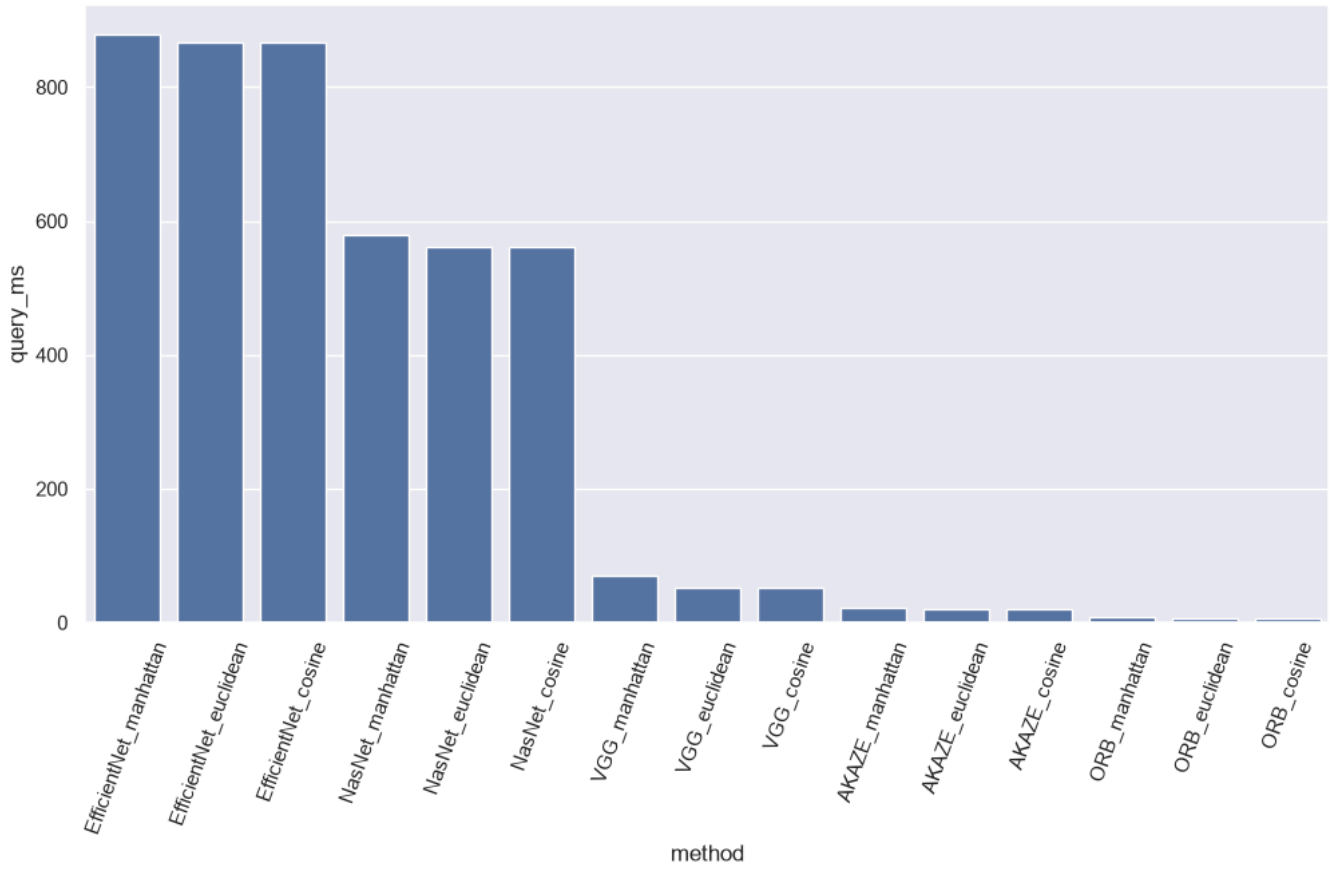
Colors & Styles

method	map@5	mrr@5	first_rank	query_ms
AKAZE_cosine	0.16409	0.375213	0.274082	19.695101
AKAZE_euclidean	0.126223	0.31519	0.225652	19.701567
AKAZE_manhattan	0.174307	0.384575	0.281533	21.518462
ORB_cosine	0.139508	0.35565	0.256519	5.591079
ORB_euclidean	0.114307	0.317279	0.219532	5.608884
ORB_manhattan	0.148775	0.363704	0.259447	7.353868
VGG_cosine	0.567769	0.789587	0.695051	52.552482
VGG_euclidean	0.537861	0.770858	0.67669	52.806309
VGG_manhattan	0.542065	0.772845	0.678819	69.171145
NasNet_cosine	0.506688	0.749197	0.651676	561.878157
NasNet_euclidean	0.499335	0.743844	0.647951	562.096291
NasNet_manhattan	0.49907	0.742962	0.649015	579.608441
EfficientNet_cosine	0.548111	0.776699	0.689729	867.427039
EfficientNet_euclidean	0.542921	0.776818	0.690261	867.572621
EfficientNet_manhattan	0.538622	0.772423	0.68281	878.440194


map@5
0.0
0.1
0.2
0.3
0.4
0.5
VGG_cosine
EfficientNet_cosine
EfficientNet_euclidean
VGG_manhattan
EfficientNet_manhattan
VGG_euclidean
NasNet_cosine
NasNet_euclidean
NasNet_manhattan
AKAZE_manhattan
AKAZE_cosine
ORB_manhattan
ORB_cosine
AKAZE_euclidean
ORB_euclidean
method


mrr@5
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
VGG_cosine
EfficientNet_euclidean
EfficientNet_cosine
VGG_manhattan
EfficientNet_manhattan
VGG_euclidean
NasNet_cosine
NasNet_euclidean
NasNet_manhattan
AKAZE_manhattan
AKAZE_cosine
ORB_manhattan
ORB_cosine
ORB_euclidean
AKAZE_euclidean
method


first_rank
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
VGG_cosine
EfficientNet_euclidean
EfficientNet_cosine
EfficientNet_manhattan
VGG_manhattan
VGG_euclidean
NasNet_cosine
NasNet_manhattan
NasNet_euclidean
AKAZE_manhattan
AKAZE_cosine
ORB_manhattan
ORB_cosine
AKAZE_euclidean
ORB_euclidean
method


query_ms
0
200
400
600
800
EfficientNet_manhattan
EfficientNet_euclidean
EfficientNet_cosine
NasNet_manhattan
NasNet_euclidean
NasNet_cosine
VGG_manhattan
VGG_euclidean
VGG_cosine
AKAZE_manhattan
AKAZE_euclidean
AKAZE_cosine
ORB_manhattan
ORB_euclidean
ORB_cosine
method

Conclusion

Les réseaux de neurones surpassent nettement les descripteurs : sur la couleur et le style ensemble, leur MAP@5 va de 0,50 à 0,57, contre 0,11 à 0,17 pour AKAZE et ORB.

Avec un classement correct, la distance cosinus n'est plus la moins bonne : elle obtient le meilleur MAP@5 sur la couleur et le style pour chaque réseau, avec des écarts faibles (moins de 0,03) face aux distances euclidienne et de Manhattan. Pour AKAZE et ORB, cosinus et Manhattan font jeu égal, devant l'euclidienne.

VGG16 avec la distance cosinus donne le meilleur MAP@5 sur la couleur et le style (0,568) et sur la couleur seule (0,591), pour environ 53 ms par requête. Sur le style seul, tous les réseaux dépassent 0,92 de MAP@5 ; EfficientNetB7 est le meilleur (0,968), mais environ 16 fois plus lent que VGG16 (environ 867 ms par requête). La couleur reste la principale source d'erreur.

Le temps d'une requête vient presque entièrement de l'extraction. La recherche exacte sur 7 627 images coûte moins de 1 ms en distance cosinus ou euclidienne, et 11 à 18 ms en distance de Manhattan pour les réseaux.